0138. Copy List with Random Pointer

| Category: Linked List | LeetCode: https://leetcode.com/problems/copy-list-with

Problem

- Each node has next and random (any node or null). Return a deep copy.

Intuition

- The hard part is that random may point forward to a node not copied yet.
- Two passes with a map original → copy: first create all copies, then wire next and random through the map.
- O(1)-space variant: interleave copies (A, A', B, B', ...), set random via orig.random.next, then unweave.

Brute Force

- For each random, search by position from the head: O(n^2).

Optimal Approach

- Pass 1: map.put(node, new Node(node.val)).
- Pass 2: copy.next = map.get(node.next); copy.random = map.get(node.random).
- Return map.get(head).

Worked Example

```
[[7,null],[13,0],[11,4],[10,2],[1,0]] (val, random index).

pass 1: 7' 13' 11' 10' 1'
pass 2: 13'.random = map(7) = 7'
        11'.random = map(1) = 1'
        10'.random = 11' ; 1'.random = 7'
```

Java Solution

```java
public class CopyListWithRandomPointer {
    public static class Node {
        public int val;
        public Node next, random;
        public Node(int val) { this.val = val; }
    }

    public Node copyRandomList(Node head) {
        for (Node n = head; n != null; n = n.next.next) {
            Node c = new Node(n.val);
            c.next = n.next;
            n.next = c;
        }
        for (Node n = head; n != null; n = n.next.next)
            if (n.random != null) n.next.random = n.random.next;
        Node dummy = new Node(0), tail = dummy;
        for (Node n = head; n != null; n = n.next) {
            tail.next = n.next;
            tail = tail.next;
            n.next = n.next.next;
        }
        return dummy.next;
    }

    public Node copyWithMap(Node head) {
        Map<Node, Node> map = new HashMap<>();
        for (Node n = head; n != null; n = n.next) map.put(n, new Node(n.val));
        for (Node n = head; n != null; n = n.next) {
            map.get(n).next = map.get(n.next);
            map.get(n).random = map.get(n.random);
        }
        return map.get(head);
    }
}
```

Complexity

- Time: O(n). Space: O(n) for the map (O(1) with interleaving).

Pitfalls

- Map.get(null) returns null, which conveniently handles null pointers.